

Task 23 – SQL Window Functions Intro

Tool Used: MySQL Workbench **Database:** Northwind

1. Objective

The objective of this task was to understand and apply SQL Window Functions to answer practical business questions. The task focused on ROW_NUMBER(), RANK(), DENSE_RANK(), and LAG(), along with PARTITION BY, GROUP BY, CTEs, and date/value comparisons.

2. Dataset

The Northwind sample database was loaded into MySQL Workbench and validated before running the analysis queries.

Table	Records
Customers	29
Products	45
Orders	48
Order Details	58

3. Tools and Technologies

- MySQL Workbench
- SQL
- Northwind sample database

4. SQL Concepts Practiced

ROW_NUMBER() – Assigns sequential numbers to rows based on a specified order.

RANK() – Assigns ranks and gives tied values the same rank, with gaps after ties.

DENSE_RANK() – Assigns ranks without gaps after tied values.

LAG() – Retrieves a value from a previous row for comparison.

PARTITION BY – Performs window calculations separately for each group.

CTE – Helps organize multi-step SQL analysis.

5. Queries Completed

Query	Technique	Business Purpose
1	ROW_NUMBER()	Number orders chronologically.
2	ROW_NUMBER() + PARTITION BY	Number each customer's orders separately.
3	ROW_NUMBER()	Find the latest order for each customer.
4	RANK()	Rank products by list price.
5	RANK() + GROUP BY	Rank customers by order count.
6	RANK() + PARTITION BY	Rank products within each category.
7	RANK() / DENSE_RANK()	Compare ranking behavior when values are tied.
8	LAG()	Show each customer's previous order date.
9	LAG() + DATEDIFF()	Calculate days since the previous order.
10	LAG()	Compare current and previous order amounts.
11	LAG() + PARTITION BY	Compare each customer's current and previous order.
12	LAG() + CASE	Classify order trends.

6. Business Analysis Performed

The queries were designed to identify the sequence of customer orders, find each customer's latest order, rank products and customers, compare products within categories, check previous order dates, calculate days between orders, compare order values, and identify whether an order value increased, decreased, or stayed the same.

7. Key Learnings

- Window Functions allow calculations across related rows while keeping individual row details.
- ROW_NUMBER() is useful for sequencing and selecting the first/latest record.
- RANK() and DENSE_RANK() are useful for ranking and handling ties.
- LAG() is useful for previous-record comparisons.
- PARTITION BY allows analysis independently for each group.
- CTEs make multi-step SQL analysis easier to structure.

8. Conclusion

Task 23 was completed successfully using MySQL Workbench and the Northwind dataset. A total of 12 SQL queries were completed, covering row numbering, ranking, dense ranking, previous-row comparisons, time-gap analysis, and business trend classification.

9. Submission Checklist

- ✓ 12 SQL queries completed
- ✓ Query outputs generated
- ✓ Notes/explanations prepared
- ✓ GitHub README prepared
- ✓ LinkedIn post prepared
- ✓ Task explanation report prepared